Module 0: Introduction

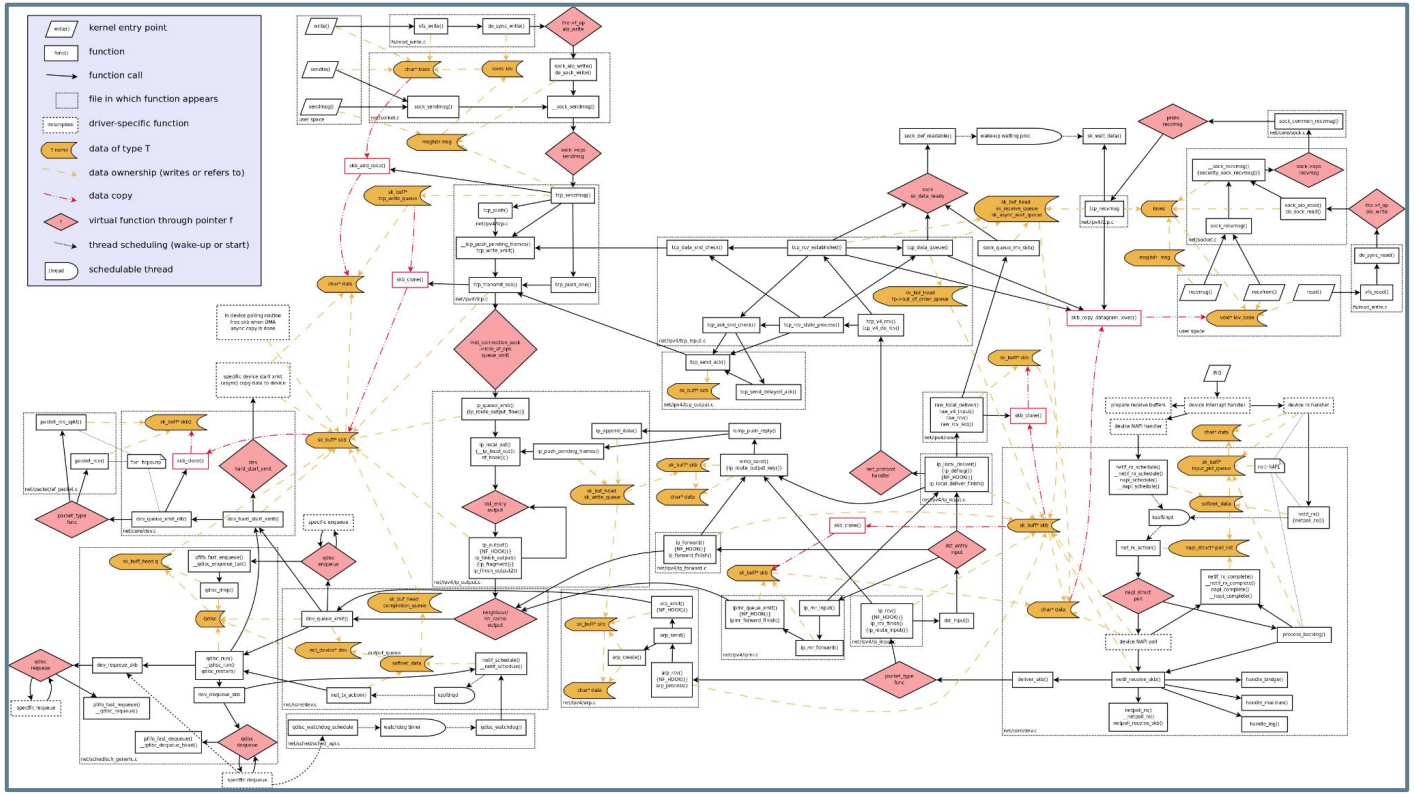
Design by Contract

Also known as: Computational Thinking 101

1.

Software Design

The least taught part of software engineering…
…and the most important.



Managing Complexity

Pterry the Terminology Pterodactyl

Definitions

What is Software Design?

The process of defining the **specification** and **structure** of a program from an outside perspective.

- API vs Implementation, Client vs Supplier, Caller vs Callee
- What is the program's **interface**?
  - This includes internal interfaces! (e.g classes - **encapsulation**)

Elements of Software Design

What are the "big ideas" to consider?

- Concepts: **complexity**, abstraction, encapsulation, modularity
- Goals: **maintainability**, extensibility, reliability, robustness, security
- Design Patterns: factory, builder, singleton, command, observer
  - Some languages have these built-in!

COP-4331 (OOP) dives into these more…

To what extent does the
programming language
affect software design?

2.

Design by Contract

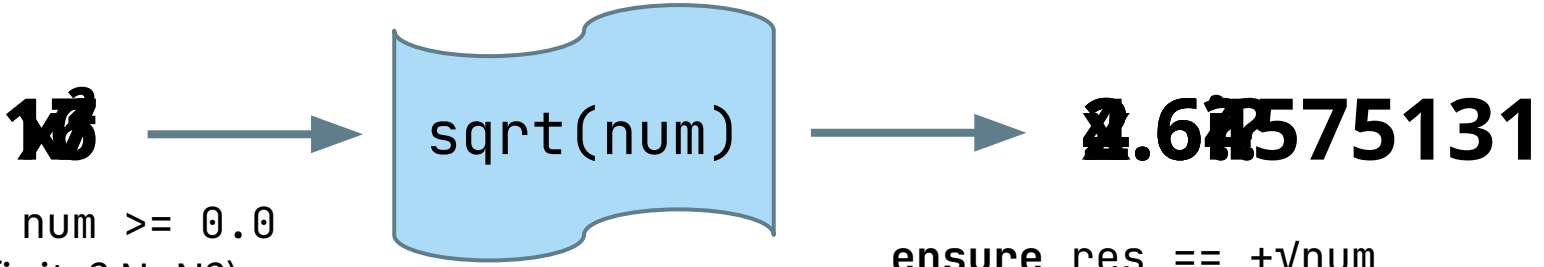
Yes, I'm reusing the title of the lecture - sue me

Contracts 101

Contracts formalize the API of functions.

A **contract** is a specification of a function (or class, etc.) including:

- **preconditions**, which are required to be met by the caller.
- **postconditions**, which are ensured by the implementation.



`require num >= 0.0`
(`-0`? Infinity? NaN?)

`ensure res == +√num`
(to some precision)

Design By Contract

Client/Supplier (Conditions)

- The **client** is responsible for satisfying preconditions
- The **supplier** is responsible for satisfying postconditions…
  - …if preconditions are satisfied!
- **Both can expect the other fulfills their part of the contract…**
  - …but can still verify as needed!

Class (State) Invariants

- Mutable data (classes) has **invariants,** which defines "valid" state
  - both precondition & postcondition
- Invariants are "always true", but may be temporarily broken within methods
  - e.g. updating two fields, red-black trees

Eiffel Design by Contract

```java
public class Account {

    /** postcondition: balance == ??? */
    public Account(...) { … }

    /** invariant: balance >= 0 (?) */
    private double balance;

    /**
     * precondition: amount > 0
     * alternate precondition: amount >= -balance (withdraw)
     * postcondition: balance += amount
     */
    public void deposit(double amount) { … }

}
```

What is the contract of this class?

What is the contract of %?

```
 7 % 3 == 1
-7 % 3 == ?
```

The Psuedocode Problem

% Operator	Remainder: C++, Java
	`7 % 3 == 1`
	`-7 % 3 == -1`
		Modulo: Python, Ruby, Perl
		`7 % 3 == 1`
		`-7 % 3 == 2`

Indexing	Zero: C++, Java `first = array[0]`	One: Matlab, Lua `first = array[1]`
Ranges	Inclusive: `1..10` (Ruby) `1...10` (Swift) `range(1, 10)` (PHP)	Exclusive: `1...10` (Ruby) `1..<10` (Swift) `range(1, 10)` (Python)

Syntax Across Languages

```java
/**
 * precondition: elements are sortable
 *  - Comparable<T>.compareTo, or add a comparator argument
 *  - don't be JavaScript!: [1, 5, 10].sort() == [1, 10, 5] (toString)
 * precondition: the list is mutable!
 *
 * postcondition: elements is sorted, or more formally:
 *  - for all indices i, j; i <= j -> elements[i] <= elements[j]
 *
 * question: what about null? (start/end? error?)
 * question: what about equal elements ('stable' sort)
 * question: what about time/space complexity? (avoid O(n^2) algorithms)
 */
public void sort(List<Object> elements) { … }

// Javadocs: Collections.sort
```

What is the contract of this function?

Watch for differences across languages!

even terminology is suspect…

3.

Testing Methodology

Writing good tests is as easy as 1, 2.0, "3".

Testing 101

Test Coverage

Coverage is a metric for how well test cases "cover" the behavior of code.

- Helpful for identifying untested behavior and edge cases.
- Provides an estimate of how well the software is tested.
- Must consider coverage in context!
  - Adding coverage has costs too.
  - 90% coverage could mean 0% of the 10% that's actually valuable.

Equivalence Partitioning

Separate test input into categories that are expected to behave the same (per spec).

- Testing all inputs is often unfeasible.
  - `f(int)`: 2^32 integer values!
  - `f(int, int)`: 2^32 * 2^32 = 2^64
  - add in classes, state, side effects, etc…
- Instead, test a subset of values that cover all **equivalence classes**, i.e. unique behaviors.
  - Start with a baseline test (bare minimum)
  - Add variations on the baseline to cover new eq. classes with *minimal changes*.

FizzBuzz Equivalence Classes

Divisible by 3 <u>only</u>	3, 6, 9, 12, ~~15~~, 18, …	"Fizz"
Divisible by 5 <u>only</u>	5, 10, ~~15~~, 20, 25, 30, …	"Buzz"
Divisible by 3 <u>and</u> 5	15, 30, 45, 60, 75, …	"FizzBuzz"
All other numbers	1, 2, ~~3~~, 4, ~~5~~, ~~6~~, 7, 8, …	number

Edge Cases

Definitions

Edge cases are specific equivalence classes (or often just values) that have distinct behavior from others.

- Typically applies to a smaller set of values that is less likely to occur in “normal” programs.
- Also applies to “combinations” of values that lead to unique cases.
  - multiply two negatives = postive

Identification

Edge cases often fall into a few categories:

- Values with unique behavior for the type.
  - null, 0, empty strings/lists
- Boundary values: specific values at the edge of valid/invalid input, typically ranges.
- Error cases: known error conditions.
  - invalid user input, IO exceptions
- Extreme input: input intentionally designed to be difficult, tricky, or even malicious.

Also, look for specific call-outs in specifications or control flow variations in implementations.

FizzBuzz Edge Cases

`0, 1`	Boundary values (lower bound) 0/1 are also common edge cases (why?)
`100, 101`	Boundary values (upper bound)
`-1`	Boundary values (negative numbers) -1 is also a common edge case (why?)
`int min/max`	Boundary values (lower bound) INT_MIN is an interesting edge case (why?)

JUnit Demo: FizzBuzz!

if time allows…

Module 0: Introduction

Design by Contract

Also known as: Computational Thinking 101

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals